A 28nm 20.9-137.2 TOPS/W Output-Stationary SRAM Compute-in-Memory Macro Featuring Dynamic Look-ahead Zero Weight Skipping and Runtime Partial Sum Quantization

Xiaofeng Hu*[1], HanGyeol Mun*[1], Jian Meng[1], Yuan Liao[1], Amitesh Sridharan[2], Jae-sun Seo[1]
[1]Cornell Tech, New York, NY [2]Arizona State University, Tempe, AZ
*Equally Credited Authors (ECAs)

SRAM-based compute-in-memory (CIM) designs have shown impressive energy efficiency for vector-matrix multiplications of AI workloads [1–9]. As many prior CIM works primarily focus on macro-level energy-efficiency, there are often big gaps between macro vs. system energy-efficiencies in real applications [10, 11]. Conventional SRAM CIM macros naturally adopt the weight-stationary (WS) dataflow, and computed partial sums are communicated to SRAM buffers outside of the CIM macro. This largely degrades the WS-CIM system energy-efficiency, especially due to frequent high-precision partial sum (Psum) read/write operations (Fig. 1). Further, while weight sparsity in AI models can be high, CIM macros with WS dataflow make zero weight skipping infeasible due to shared activation and static weight storage. Previous works only support limited structured (e.g. block-wise) sparsity [4], or only store non-zero weights with indices but do not have latency benefits [7]. Several systolic array designs presented skipping techniques for sparse matrices [12–14], but they all require large registers and complex connections, which are unsuitable for compact CIM macros. To address these challenges, this paper presents the first output-stationary SRAM digital CIM (OS-CIM) macro and system, featuring (1) an OS-CIM structure with custom single-cycle read-and-write 8T SRAM cell to store the Psum in the CIM macro and perform the accumulation locally, eliminating the expensive Psum access to SRAM buffers, (2) a dynamic look-ahead weight skipping (DLS) scheme, well aligned with OS-CIM, which achieves high latency reduction across various weight sparsity patterns and significantly improves workload balance, and (3) a runtime Psum quantization (RPQ) method to mitigate the hardware cost of high-precision accumulation.

Fig. 2 presents the overall OS-CIM system, consisting of two 16-kb SRAM OS-CIM macros that support 4b/8b MAC operations, 8-kb activation SRAM buffer, 8-kb weight SRAM buffer, and top controller. Every OS-CIM macro is divided into 16 groups, where each group has a WL driver, DLS control, RPQ control, and 4×256b 8T OS-CIM cells. The four OS-CIM rows in the same CIM group share the local multiplier (which multiplies bit-serial activation and 4b/8b weights) and the Psum accumulator. Unlike the conventional WS-CIM, the proposed OS-CIM stores the Psum in custom 8T OS-CIM SRAM cells. By selectively asserting read wordline (RWL) or write wordline (WWL), the cell can perform read, write, or same-cycle read and write. The local accumulator is reconfigurable to support 8b/12b/16b Psum precision. DLS controls the zero weight skipping with the weight buffer by selectively activating the RWL/WWL driver for the OS-CIM cell row with non-zero weights. The RPQ module monitors the accumulation results and shifts the dynamic range accordingly when overflow is detected. Weight shifter obtains the weight from DLS and shifts it according to the current activation bit index and the shared exponent of RPQ.

Fig. 3 shows the detailed dataflow of OS-CIM with the example of 8b activation/weight (A/W) precision. After initiating the cell to zero, the Psum keeps accumulating locally in the OS-CIM with bit-serial activation feeding until the current MAC is finished and the result will be read out. For the single-cycle MAC operation, when clock is high, RWL is turned on and WWL is turned off for the selected row, and the Psum is read out. This is accumulated with the current bit-wise product, and when clock is low, the updated Psum is written back to the same cell in the same cycle by turning off RWL and turning on WWL without overlap. The shifted weight is shared horizontally with all cells in the same row, and as each activation bit is fed vertically, every row computes the elements of the same output channel in parallel. Sign-magnitude format is more suitable than two's complement for bit-serial computation because it significantly lowers the toggling activity rate. If the activation bit is zero, Psum accumulation/writeback operations are not executed at all. Compared to two's complement, the number of dynamic MAC and writeback operations with sign-magnitude format is reduced by 32.1%/22.6% in ResNet34/ViT-Base inference. With such optimization and the large reduction of Psum buffer access, the overall system energy of OS-CIM is improved by 26-35% than the WS-CIM scheme for inference of ViT-Base and ResNet-34 models.

Fig. 4 illustrates the DLS module that skips the zero weights. The input weight index is decoded to a 4×4 bitmask for DLS to check the non-zero weight position. The A/W selection signals are generated by the current pointer ('Cur_ptr') and sent to WL driver and OS-CIM cell. The leading zero counter checks the bitmask and detects the pointer of next non-zero weight (denoted by 'Next_ptr'). The 'Look_ahead' signal of each CIM group is set to 1 if the current column ('Cur_Col') is moved beyond the first. The look-ahead buffer window moves forward column-wise when 'Look_ahead' signals of all groups become 1, upon which the next A/W set will be loaded. An example OS-CIM operation with 50% random weight sparsity is illustrated in Fig. 4. Without DLS, all CIM groups need to wait for the slowest group to finish its computation, after which the next activation can be fed, and it takes 15 cycles to finish the example computation. With the proposed DLS scheme, when four activations are fed at the same time, DLS allocates more tasks to each group, and thus the inter-group workload balance is significantly improved. As a result, the computation is completed in just 8 cycles. For the ViT-Based model, DLS boosts the system utilization and throughput by 1.9-5.4× and 1.3-3.6× across unstructured [21] and N:M structured [19] weight sparsity schemes, compared to WS-CIM.

Fig. 5 shows the RPQ mechanism to mitigate the hardware cost of high-precision Psum accumulation. By changing the carry in (Ci) signal connection of 4b FAs, the local accumulators can be configured to support 8b/12b/16b Psum precision. Psums in the same row share a block exponential offset, stored in 4b 'Shared EXP(S-Exp) Reg'. The accumulation starts with S-Exp value of 0. When an overflow is detected, S-Exp increases by 1, and all Psum results in the current row will be right-shifted by 1 to match the dynamic range. The weight is also shifted according to the S-Exp value of the current Psum row. During readout, S-Exp and Psum mantissa in the OS-CIM cell are output together to recover the original value. We developed a cycle-accurate hardware simulator that evaluates the end-to-end model accuracy with our RPQ scheme. The simulator has been verified on off-the-shelf INT8 models from Torchvision [20] and various algorithms with quantization/sparsity [15–19]. RPQ improved energy efficiency and throughput by 24-28% and 50-59% on model evaluation, with negligible accuracy loss through adaptive Psum precision selection according to the layer-wise Psum quantization sensitivity.

A 32-kb OS-CIM macro with 16-kb off-the-shelf SRAM on-chip buffer is fabricated in 28nm CMOS (Fig. 7). Fig. 6 shows the chip measurement results. The OS-CIM system operates at 0.45-1.0V supply and 33-574MHz frequency, while consuming 0.91-68mW power. Peak energy-efficiency of 137.2 TOPS/W is achieved at 0.45V with 2:16 sparsity and INT8 precision for inference of ViT-Base model. Compared to [4] with block-wise sparsity of ∼35%, our work achieves the same ResNet50 accuracy for ImageNet-1K with 85% unstructured sparsity, while achieving 1.45× higher system-level energy efficiency. While [6] reports comparable system-level TOPS/W with a systolic structure, its larger chip area improves data reuse, which helped enhance the system energy-efficiency. For an apple-to-apple comparison, we scaled up our system to match the area of [6] including SRAM buffers and CIM macros, and our OS-CIM system achieved 1.39× higher system energy-efficiency. Compared to state-of-the-art DCIM works, our work achieves among the highest macro-level TOPS/W, but more importantly also achieves high ratio of system-level vs. macro-level energy-efficiency, as well as latency reduction corresponding to the weight sparsity ratio.

Acknowledgements: This work is supported in part by the CoCoSys center in JUMP 2.0, an SRC program sponsored by DARPA.

2025 IEEE Custom Integrated Circuits Conference (CICC) | 979-8-3315-1745-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/CICC63670.2025.10982878

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.
979-8-3315-1745-8/25/$31.00 ©2025 IEEE

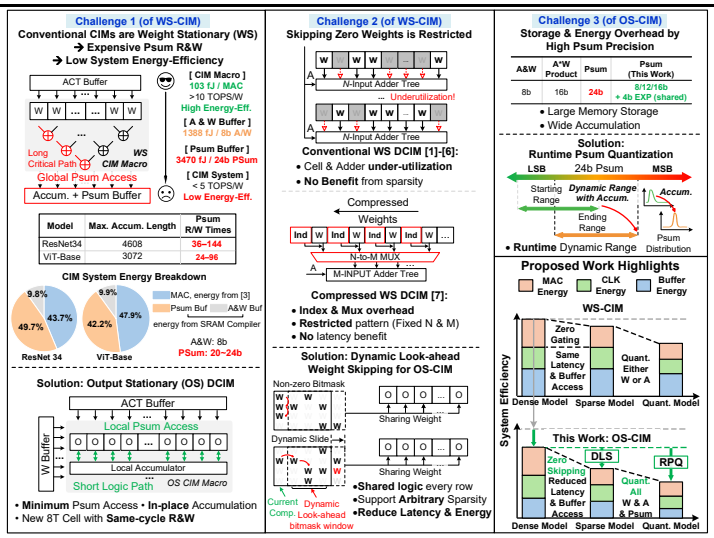


Fig. 1. Challenges of conventional CIM designs and our proposed solutions.

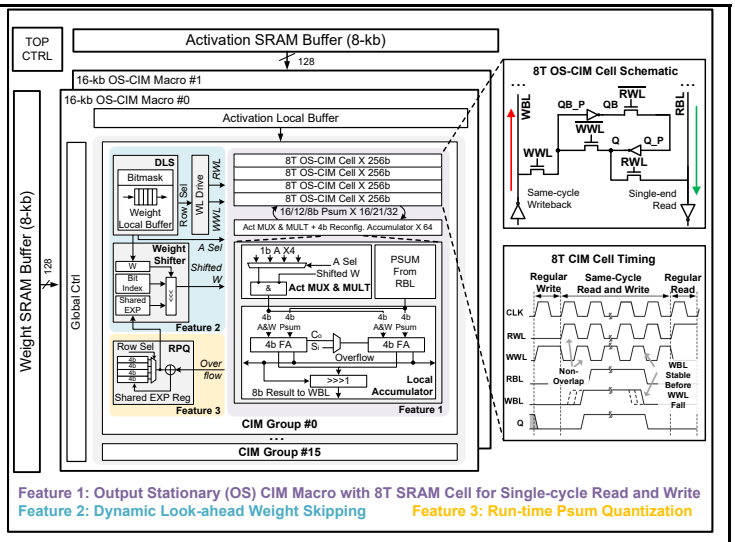


Fig. 2. Overall architecture of proposed output stationary (OS) CIM macro and system.

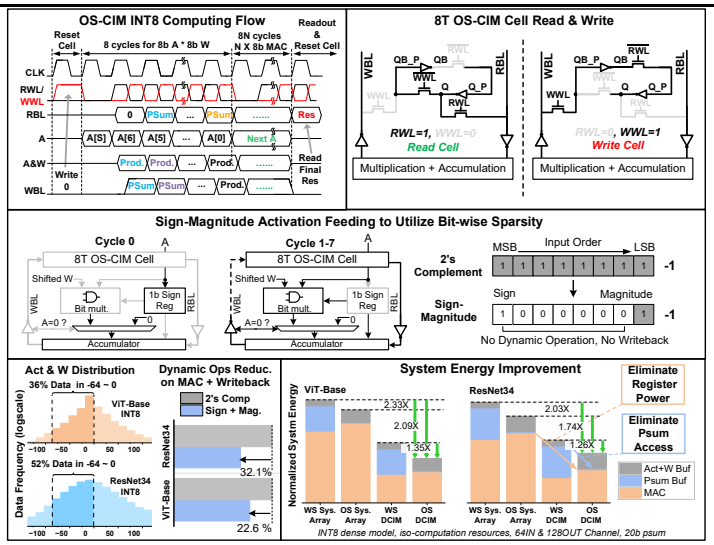


Fig. 3. Computing flow of OS-CIM with single-cycle partial sum read, accumulation, and write operations.

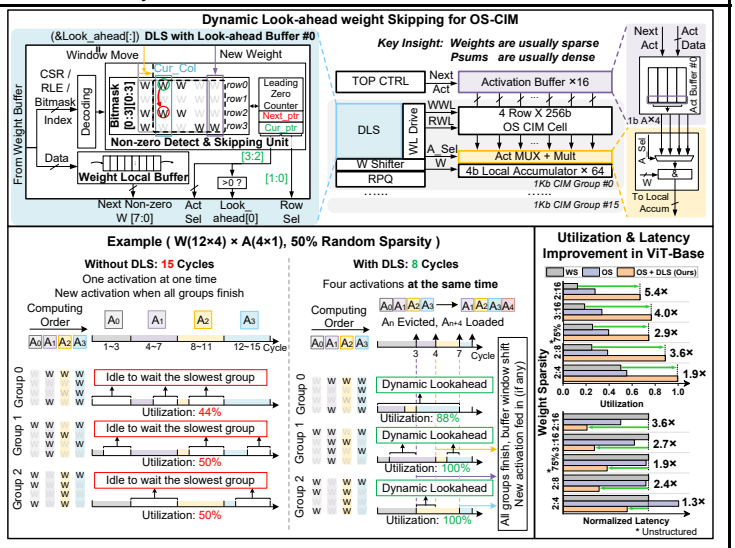


Fig. 4. Operation of dynamic look-ahead weight skipping (DLS) scheme that skips the zero weights and reduces latency of OS-CIM macro accordingly.

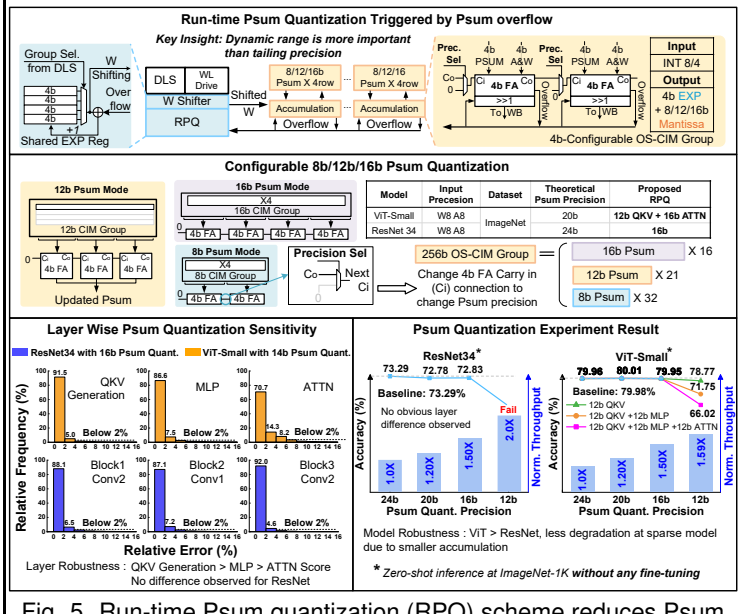
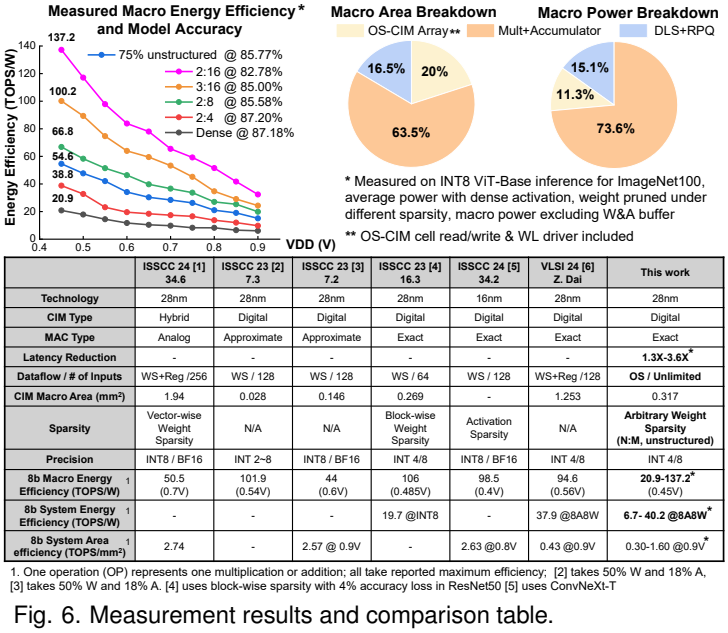


Fig. 5. Run-time Psum quantization (RPQ) scheme reduces Psum precision (from 24b to 16b for 8b activations/weights) in OS-CIM without degrading DNN/ViT accuracy.

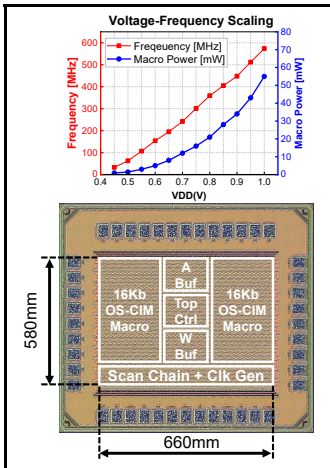


	ISSCC 24 [1] 34.6	ISSCC 23 [2] 7.3	ISSCC 23 [3] 7.2	ISSCC 23 [4] 16.3	ISSCC 24 [5] 34.2	VLSI 24 [6] Z. Dai	This work
Technology	28nm	28nm	28nm	28nm	16nm	28nm	28nm
CIM Type	Hybrid	Digital	Digital	Digital	Digital	Digital	Digital
MAC Type	Analog	Approximate	Approximate	Exact	Exact	Exact	Exact
Latency Reduction	-	-	-	-	-	-	**1.3X-3.6X***
Dataflow / # of Inputs	WS+Reg /256	WS / 128	WS / 128	WS / 64	WS / 128	WS+Reg /128	**OS / Unlimited**
CIM Macro Area (mm²)	1.94	0.028	0.146	0.269	-	1.253	0.317
Sparsity	Vector-wise Weight Sparsity	N/A	N/A	Block-wise Weight Sparsity	Activation Sparsity	N/A	**Arbitrary Weight Sparsity (N:M, unstructured)**
Precision	INT8 / BF16	INT 2~8	INT8 / BF16	INT 4/8	INT8 / BF16	INT 4/8	INT 4/8
8b Macro Energy Efficiency (TOPS/W) [1]	50.5 (0.7V)	101.9 (0.54V)	44 (0.6V)	106 (0.485V)	98.5 (0.4V)	94.6 (0.56V)	**20.9-137.2*** (0.45V)
8b System Energy Efficiency (TOPS/W) [1]	-	-	-	19.7 @INT8	-	37.9 @8A8W	**6.7- 40.2 @8A8W***
8b System Area Efficiency (TOPS/mm²) [1]	2.74	-	2.57 @ 0.9V	-	2.63 @0.8V	0.43 @0.9V	0.30-1.60 @0.9V*

1. One operation (OP) represents one multiplication or addition; all take reported maximum efficiency; [2] takes 50% W and 18% A, [3] takes 50% W and 18% A. [4] uses block-wise sparsity with 4% accuracy loss in ResNet50 [5] uses ConvNeXt-T

Fig. 6. Measurement results and comparison table.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.



		Chip Summary
Technology		28nm CMOS
CIM Type		Digital
Working Voltage		0.45V-1.0V
Cell Structure		8T (Logic rule)
CIM Macro size		32Kb
SRAM size		16Kb
Macro Area (mm²)		0.317
Input Channels		Unlimited
Output Channels		64
Sparsity		Arbitrary
Precision		INT 4/8
Macro energy efficiency[1] (TOPS/W)		20.9-137.2
System energy efficiency [1] (TOPS/W)		6.7-40.2
System area efficiency (TOPS/mm²)		0.30-1.60 (0.9V)
Chip[2] Inference Accuracy	ViT-Small	78.77% (↓1.22%)
	ResNet34	72.80% (↓0.49%)

1. Average case, from dense to 87.5% sparse ViT-Base model at 0.45V
2. Dataset: ImageNet-1K

Fig. 7. Chip micrograph and performance summary.

References:

[1] Y. Yuan et al., "34.6 A 28nm 72.12TFLOPS/W Hybrid-Domain Outer-Product Based Floating-Point SRAM Computing-in-Memory Macro with Logarithm Bit-Width Residual ADC," ISSCC, 2024. https://doi.org/10.1109/ISSCC49657.2024.10454313

[2] Y. He et al., "7.3 A 28nm 38-to-102-TOPS/W 8b Multiply-Less Approximate Digital SRAM Compute-In-Memory Macro for Neural-Network Inference," ISSCC, 2023. https://doi.org/10.1109/ISSCC42615.2023.10067305

[3] A. Guo et al., "A 28nm 64-kb 31.6-TFLOPS/W Digital-Domain Floating-Point-Computing-Unit and Double-Bit 6T-SRAM Computing-in-Memory Macro for Floating-Point CNNs," ISSCC, 2023. https://doi.org/10.1109/ISSCC42615.2023.10067260

[4] J. Yue et al., "A 28nm 16.9-300TOPS/W Computing-in-Memory Processor Supporting FloatingPoint NN Inference/Training with Intensive-CIM Sparse-Digital Architecture," ISSCC, 2023. https://doi.org/10.1109/ISSCC42615.2023.10067779

[5] W.-S. Khwa et al., "34.2 A 16nm 96Kb Integer/Floating-Point Dual-Mode-Gain-Cell-Computing-in-Memory Macro Achieving 73.3-163.3TOPS/W and 33.2-91.2TFLOPS/W for AI-Edge Devices," ISSCC, 2024. https://doi.org/10.1109/ISSCC49657.2024.10454447

[6] Z. Dai et al., "A 41.7TOPS/W@INT8 Computing-in-Memory Processor with Zig-Zag Backbone-Systolic CIM and Block/Self-Gating CAM for NN/Recommendation Applications," Symp. VLSI, 2024. https://doi.org/10.1109/VLSITechnologyandCir46783.2024.10631368

[7] A. Sridharan et al., "SP-IMC: A Sparsity Aware In-Memory-Computing Macro in 28nm CMOS with Configurable Sparse Representation for Highly Sparse DNN Workloads," CICC, 2024. https://doi.org/10.1109/CICC60959.2024.10529009

[8] C. Yu et al., "A Dual 7T SRAM-Based Zero-Skipping Compute- In-Memory Macro With 1-6b Binary Searching ADCs for Processing Quantized Neural Networks," TCAS-I, vol. 71, no. 8, Aug. 2024 https://doi.org/10.1109/TCSI.2024.3411608

[9] M. Yang et al., "CILP: An Arbitrary-bit Precision All-digital Compute-in-memory Solver for Integer Linear Programming Problems," CICC, 2024. https://doi.org/10.1109/CICC60959.2024.10529041

[10] X. Sun et al., "Efficient Processing of MLPerf Mobile Workloads Using Digital Compute-In-Memory Macros," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 43, no. 4, Apr. 2024. https://doi.org/10.1109/TCAD.2023.3333290

[11] H. E. Sumbul et al., "A Fully Digital and Row-Pipelined Compute-in-Memory Neural Network Accelerator With System-on-Chip-Level Benchmarking for Augmented/Virtual Reality Applications," IEEE Micro, vol. 44, no. 2, Mar.-Apr. 2024. https://doi.org/10.1109/MM.2023.3338059

[12] J.-F. Zhang et al., "SNAP: An Efficient Sparse Neural Acceleration Processor for Unstructured Sparse Deep Neural Network Inference," JSSC, Feb. 2021. https://doi.org/10.1109/JSSC.2020.3043870

[13] X. He et al., "Squaring the circle: Executing Sparse Matrix Computations on FlexTPU—A TPU-Like Processor," PACT, 2022. https://doi.org/10.1145/3559009.3569665

[14] X. He et al., "Sparse-TPU: adapting systolic arrays for sparse matrices," ICS, 2020. https://doi.org/10.1145/3392717.3392751

[15] J. Meng et al., "Torch2Chip: An End-to-end Customizable Deep Neural Network Compression and Deployment Toolkit for Prototype Hardware Accelerator Design," MLSys, 2024. https://doi.org/10.48550/arXiv.2405.01775

[16] M. Nagel et al., "Up or Down? Adaptive Rounding for Post-Training Quantization," ICML, 2020. https://doi.org/10.48550/arXiv.2004.10568

[17] S. Siddegowda et al., "Neural Network Quantization with AI Model Efficiency Toolkit (AIMET)," Arxiv, 2022. https://doi.org/10.48550/arXiv.2201.08442

[18] X. Wei et al., "QDrop: Randomly Dropping Quantization for Extremely Low-bit Post-Training Quantization," ICLR, 2022. https://doi.org/10.48550/arXiv.2203.05740

[19] A. Zhou et al., "Learning N:M Fine-grained Structured Sparse Neural Networks From Scratch," ICLR, 2021. https://doi.org/10.48550/arXiv.2102.04010

[20] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," ArXiv, 2019. https://arxiv.org/abs/1912.01703

[21] S. Liu et al., "Sparse Training via Boosting Pruning Plasticity with Neuroregeneration," NeurIPS, 2021. https://dl.acm.org/doi/10.5555/3540261.3541019

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 29,2026 at 12:09:40 UTC from IEEE Xplore. Restrictions apply.